Microservices as Agents in IoT Systems

Petar Krivic(✉), Pavle Skocir, Mario Kusek, and Gordan Jezic

Internet of Things Laboratory, Faculty of Electrical Engineering and Computing, University of Zagreb, Unska 3, HR-10000 Zagreb, Croatia
{petar.krivic,pavle.skocir,mario.kusek,gordan.jezic}@fer.hr

Abstract. Developing robust monolith systems has achieved its limitations, since the implementation of changes in today's large, complex, and fast evolving systems would be too slow and inefficient. As a response to these problems, microservice architecture emerged, and quickly became a widely used solution. Such modular architecture is appropriate for distributed environment of Internet of Things (IoT) solutions. In this paper we present a solution for service management on Machine-to-Machine (M2M) devices within IoT system by using collaborative microservices. Collaboration of distributed modules highly reminds of multi-agent systems where autonomous agents also cooperate to provide services to the end-user. Because of these similarities we consider microservices as modern agents that could improve systems in distributed environments, such as IoT.

Keywords: IoT · M2M · Microservices · Agents · Service management

1 Introduction

Internet of Things (IoT) is not just a popular topic generating a lot of interesting scientific prototypes, but a very popular concept with a lot of existing industrial solutions in everyday use. Connecting a large amount of devices to Internet has shown a lot of benefits and possibilities, but also leaves space for improvement. It is important to offer users and companies interested in this domain a large number of different possibilities and architectures, so they could choose the one that suits them the most.

Common architecture of IoT system has a robust server node where data is processed and stored. It is also connected with M2M devices that generate this data through M2M gateway, a middle-node that translates messages of different protocols. It turned out that processing such amount of data could cause problems on standard servers and alternative was needed. Processing data in cloud is today the best alternative, but other options should be studied as well, so that producers have more options to combine and apply to their products, depending on their needs. Because of the distributed environment of IoT systems we believe that using microservices in their implementation would be an applicable approach, since they also have distributed architecture.

© Springer International Publishing AG 2018
G. Jezic et al. (eds.), *Agent and Multi-Agent Systems: Technology and Applications*, Smart Innovation, Systems and Technologies 74,
DOI 10.1007/978-3-319-59394-4_3

Lately, microservices became a very popular topic in software development. Since it was hard to maintain vast amounts of code in large companies, an idea of modularization proved to be a good alternative. Modular code is easier to upgrade, to fix if needed and new employees can get familiarized with it quickly. Also, teams could focus on parts of the system that are their responsibility without the need of closely knowing the code of the entire system. Microservice architecture has a lot of similarities with IoT systems [3], which was our motivation to implement the system described in continuation of this paper.

Distributed microservice architecture has also similarities to agent system architecture. Requests are processed collaboratively and only the end result is delivered to end-user. In order to test the behavior of microservices in IoT system, and to point out similarities between them and agents in multi-agent systems, we present a solution described in continuation.

Section 2 gives a short overview of related work in connection with IoT, microservices and agent-based systems. Section 3 presents the purpose of service management in our system and proposed architecture used to achieve it. Next, Sect. 4 describes advantages of microservice architecture and a comparison between agents and microservices is given. Finally, Sect. 5 concludes the paper.

2 Related Work

IoT systems architecture was inherited from Machine-to-Machine (M2M) concept. Since this concept is slightly older then Internet of Things there are already some existing standards in use [4] where architecture is proposed, and a great number of prototypes has already accepted it. Bhowmik et al. [2] used proposed architecture in their project intended for ambient assisted living (AAL). They connected constrained devices that perform measurements with their social network server through gateway. This is the most common case of data processing in M2M applications, where gateway forwards sensor data to server continuously.

Repetitive measurements of sensor values and forwarding them to gateway can be challenging for battery-powered devices with limited energy available. Finding more efficient solution of this problem is also a very popular research topic. In [9] a solution where interval of measurements would increase if observation values were similar is proposed, assuming that changes in observed space are not as frequent as expected. This way the amount of measurements and sending data is reduced, thereby accomplishing energy savings.

Internet of Things architecture is quite similar to microservice architecture as Butzin et al. conclude in [3]. They compared distributed architecture of IoT systems with microservices, and pointed out that both of them have the same architecture goal. Since they are both modular, their goal is to create application which unifies a large number of services. Main differences between these two are a result of processing limitations that M2M devices have. This is the reason why in some cases microservice architecture is easier to deploy and manage then it is the case with IoT solutions.

Distributed architecture of microservices suddenly became widespread, but new challenges emerged with it. In [10] Xu et al. highlight programming systems

that consist of a large number of services, reducing communication overhead in these systems and flexible deployment of services to a network as three main challenges in microservice system construction. In their work they tackle these challenges with agent-oriented programming language CAOPLE. They propose separating service as functionality from service as a computational entity that offers such functionalities. So in their novel language service is functionality offered by agent, where they point out similarity with object oriented design. Microservices in their work are then presented as agents which can communicate using some of the provided mechanisms. Also, in [5] Liu et al. present a software development environment where programming in previously mentioned CAOPLE is enabled and the user can test the execution of created application.

In this paper we demonstrate the similarities between agents and microservices. Firstly, we describe our goal with two approaches we used to achieve it. Then a comparison between two used approaches is given with a review on their compliance with popular software architecture properties. Finally, we describe similarities of microservices and agents with a conclusion why we think there is a significant resemblance between these two concepts.

3 Agent Model for Service Management in IoT Systems

In order to test agent network model based on microservices, we decided to implement a solution which would enable service management on M2M devices in our M2M network. The main goal was to achieve two-way communication with M2M devices which is then used in registration process of devices and services in our system, and to start or stop registered services based on the requirements of interested parties.

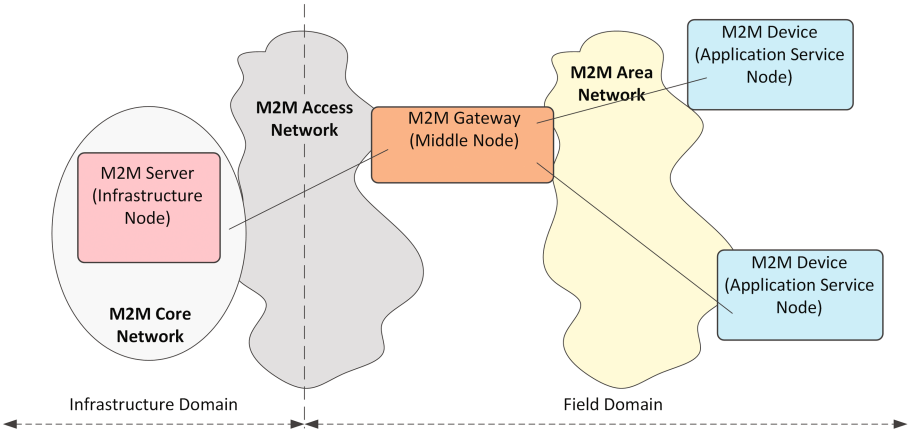

Fig. 1. oneM2M architecture

OneM2M's recommended architecture [7] for M2M systems depicted in Fig. 1 shows the environment of our solution. We used Waspmote PRO v1.2 as our M2M devices which communicate with Waspmote gateway using the XBee 802.15.4 area network. Since our goal was to enable communication between the end-user accessing our system over the Internet and M2M devices in our M2M area network, we had to implement and integrate agents in both of previously mentioned networks. In M2M area network we increased functionalities of M2M devices to make them more agent oriented, which is described in the following subsection.

However, the main focus of this paper is to present agent model in IoT system based on microservices, used for implementation of M2M server and M2M gateway in Fig. 1. Gateway is a middle node that has a role to forward messages between M2M server and M2M devices. Our M2M server consists of user-interface where user can manage services, database where all data is stored and communication interface towards gateway. Also, to compare advantages and disadvantages of proposed microservice model with other approaches, we developed a monolithic model as well. More detail description of these two models is given in continuation.

3.1 Agent Based M2M Devices

Since Internet of Things concept emerged, usual practice was to collect data from M2M devices continuously, which is not an energy efficient approach. Furthermore, in this way a lot of redundant or unnecessary data is measured, sent to network and stored in databases [1].

This could be resolved, or at least improved, if users could somehow manage existing services on M2M devices. In order to achieve this goal we extended mentioned usual practice so that our M2M devices could adapt to the end-user requirements. The idea was to create a system that could show all available services offered by reachable M2M devices. The user could then start or stop selected service, and set the time frequency of measurements in order to spare energy, but also to receive requested data in specified time intervals.

Algorithm 1 shows proposed procedure for M2M devices in M2M network that are a part of our system. Firstly, the device waits for messages from gateway, scheduled at the time of the wakeup. There are two types of messages it can receive: request for registration of services offered by device and request to start or stop the specified service. If the registration request is received, registration data is sent towards gateway. Otherwise, if start or stop request is received, device turns on or off (depending on the current state) measurements for the specified service. Also, in this case device updates interval of hibernation to the received value. Afterwards, the device performs measurements and sends the collected data if any service is active. Finally, the device goes to sleep for specified amount of time, after which the process is repeated.

Algorithm 1. Proposed M2M device algorithm

```
procedure TWO-WAY COMMUNICATING DEVICE
loop:
    receive messages from gateway.
    if message received then
        if registration request received then
            send device registration data.
        else
            if measuring and sending == ON then
                measuring and sending = OFF
            else
                measuring and sending = ON
            x = received time interval.
    if measuring and sending == ON then
        measure sensor values.
        send measured sensor values.
    sleep (x).
    goto loop.
```

3.2 Architectures Description

In our first approach we implemented a monolith M2M server that communicates using the HTTP communication protocol with M2M gateway. Our goal was to compare microservices approach with monolith architecture but because of the distributed environment in IoT, we had to develop two-component system. First component consists of data manipulation, application logic and interaction with the end-user and it is the monolithic core of our system. Second component is the gateway that forwards messages between M2M network and the mentioned core. Since in this case gateway is just a necessary separate part of the our core we decided to implement it as a simple runnable application. Thus, it can be deployed on every device which has a gateway for XBee 802.15.4 communication attached to it.

To achieve all necessary functionalities of our system using microservice architecture, we developed three microservices (Fig. 2). Two of them form the M2M server and the third is M2M gateway in Fig. 1. Since it is recommended that one microservice should be small and autonomous but also collaborative [6], we designed mentioned architecture where every microservice performs simple tasks, but in collaboration with other microservices all functionalities of our system are achieved. It is also recommended that each microservice should have its own business value. However, in distributed environment such as IoT this is hard to achieve, so in our system they offer business value only in collaboration. Communication between microservices is accomplished using the HTTP protocol, so each microservice has its own RESTful interface. Tasks of each microservice are:

- Microservice 1 (Database agent) - Stores data received from gateway to database;

– Microservice 2 (Gateway agent) - This microservice has several tasks since it communicates with 802.15.4 network. Firstly it registers itself to database agent, so database agent knows the address where gateway is available. Then, it periodically discovers reachable devices in 802.15.4 network and initiates their registration. Finally, this agent forwards all data received from M2M devices towards database agent and the other way around.
– Microservice 3 (User Agent) - This agent interacts with the end-user. It offers interface where all available devices are shown and then the end-user can manage offered services.

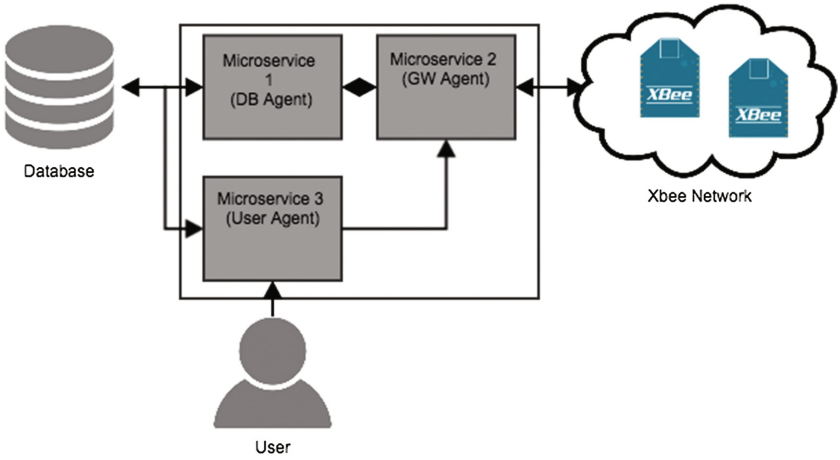

Fig. 2. Microservice architecture

4 Benefits of Microservice Approach in IoT Systems

As mentioned before, we implemented a system that enables service control on M2M devices in M2M network using both of the previously described approaches. We achieved two-way communication between the end-user and M2M devices. Consequently we achieved our goal in energy savings on M2M devices and also, we reduced the need of physical interaction between system provider and mentioned devices. System provider could now control active devices remotely which is important in cases when devices are physically hard to reach.

In continuation we present our comparison of monolith and microservice architecture, with more detail description why we propose microservice architecture over others in research oriented systems. Also, we describe how each microservice could in fact be considered as a single agent. System made of several microservices is then a multi-agent system where each agent, in collaboration with others, processes requests.

4.1 Comparison of Microservices with Monolith Approach

The greatest advantage that microservice architecture offers in comparison to other architectures is agility. One of the motives for creating microservices was to modularize complex and robust systems, in order to increase speed of deploying new solutions or upgrading the old ones [6].

We also first developed monolith system to solve previously described problem, and then decided to modularize it. Our motivation to do so were constant changes and improvements we had to do on our system, since we wanted to research different possibilities in the IoT domain. Every change would then impact the whole system and consequently we had to revise the whole code while the system would be unavailable. So in order to avoid this we decided to use microservices in future, which is suitable for research and development projects where a lot of changes are made daily, and the scope of the developing system is always increasing.

Table 1. Property comparison for each architecture

Property	Microservice architecture	Monolith architecture
Scalability	Replicating microservices across servers depending on demand	Replicating the whole monolith system on multiple servers
Development	Different programming languages in one system	All services written in same programming language
Changeability	Changing occurs on a single microservice	Changing one service causes revisioning and redeploying the entire system
Integration	Challenging because of the large number of distributed modules	Modules placed together
Communication	Defined communication interface between microservices	In-memory
Maintenance	Smaller pieces of code	One source code for the whole system
Upgrades	Adding new microservices	Upgrading the whole system

Table 1 shows some of the most popular properties that are considered when designing a new system. Scalability is easier to achieve when using microservice architecture since the owner has to replicate only microservices that are overloaded. In case of monolith architecture the owner has to replicate the complete system on multiple servers and add a strategy for load balancing between system replicas. Development is also more convenient with microservices. System architect can have heterogeneous development team regarding programming languages, and still create fully functioning solution. Every microservice can be implemented in different programming language as it is only important that the communication interface between them is well defined. Again, in monolith

architecture everything should be implemented using the same programming language. Agility and upgrades are also much easier in modular systems because changing small modules or creating and integrating new ones is much easier when they do not impact the complete system. If a single microservice has a key role in the system, it can be easily replicated and changed when the new one is compliant with communication interface used in the system. Equally, maintenance of smaller code pieces that implement modules is easier then updating robust monolithic source code.

However, monolith architecture has also its advantages as for example communication between services and integration of the system. Since all services are a part of the same system placed on a single server, they communicate using the same memory and their integration is implied. In our opinion communication could be the greatest weakness of microservice architecture, since a great amount of communication between different services occurs on every request. In most cases HTTP is not efficient enough, and alternatives are different messaging systems such as RabbitMQ, Apache Kafka etc. These messaging systems accelerate message exchange between microservices, but since in most cases there is no possibility of parallel processing there is still time overhead compared to monolith system procedure time.

4.2 Microservices as Agents

Because of the distributed architecture and other properties, microservices are in our opinion modern agents. As it can be seen in Table 2, microservice has almost every well known agent property [8]. For that reason we could present microservice architecture of our system as a collaborative multi-agent composition. Every microservice in our system is then in fact an agent which performs modest tasks, and in collaboration with other agents it delivers a fully functional service to the end-user.

Autonomy of every microservice is obvious since it can accomplish its task without human intervention and independently of other services. In Fig. 2 all three microservices could in fact be run as a stand-alone services. Microservice 2 for example has a gateway role. Once it is up, it forwards messages and performs other activities completely autonomously. Microservice 1 stores data in database, and Microservice 3 fetches data from it, so it could offer GUI to the end-user. Therefore, these could equally work autonomously.

Interaction, collaboration and coordination are three basic properties of microservices. A modular system consisting of multiple microservices must have a well defined interface between them, in order to work properly and efficiently. Every microservice in Fig. 2 would not have much purpose on its own, but together they form a complete well functioning IoT system. For example if a user wants to start a service and receive data from specified device, it can send request using Microservice 3. However, if Microservice 2 does not forward request to the device and receive data from it, or on the other hand, if Microservice 1 does not store data to database, user will not receive the wanted result.

Mobility is also easy to achieve by using microservices. System owner can run and move each microservice independently, as long as other microservices can reach it. If HTTP protocol is used for communication between them, they must have an option to update the address of moving agent. In case messaging system is used, this is also not necessary since then the moving agent only needs to maintain or establish new connection towards messaging broker.

Reactivity of microservice agents in our system is defined as their implementation logic. Once our microservices are up and running they must be aware of the data processing procedure. Also, if we consider supporting additional communication protocols on gateway (Microservice 2), reactivity is even more highlighted. In that case agent is a multi-protocol reactive bridge of the system, which is responsible for using correct procedures in the right moment.

Learning and adaptation are two agent properties which are not contained in microservices of our system. However, these properties are also achievable in microservices. They are left out from our system for now, but in future upgrades they might be considered.

Table 2. Software agent properties in microservices

Property	
Autonomy	✓
Adaptation	✗
Interaction	✓
Mobility	✓
Learning	✗
Collaboration	✓
Coordination	✓
Reactivity	✓

5 Conclusion

This paper presented benefits of microservice architecture and its similarities with agent-based architecture. In our opinion the greatest improvement brought by microservices is agility of system development. After modularizing our monolith system and shifting it to microservice architecture we could focus on parts we wanted to research, without the need of revising the whole code after every new feature. Since each microservice is independent individual that performs a single service, we presented them as agents in our systems. Each agent has its own purpose and together they form a collaboration that accomplishes the goal of the system.

In our future work we plan to implement additional agents in order to offer new services to end-users. We also plan to add learning and adoption properties

to our microservices to achieve dynamic adjustments to the environment they are placed in. Also, in this way our microservices will have all the properties of the agents. Therefore we think this will be indisputable argument that in future microservices should be used to implement agent-based systems.

Acknowledgements. This work has been supported by Croatian Science Foundation under the projects 8065 (Human-centric Communications in Smart Networks) and 8813 (Managing Trust and Coordinating Interactions in Smart Networks of People, Machines and Organizations), and by Croatian Regulatory Authority for Network Industries under the project Looking to the Future 2020.

References

1. Alduais, N.A.M., Abdullah, J., Jamil, A., Audah, L.: An efficient data collection and dissemination for IOT based WSN. In: 2016 IEEE 7th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), pp. 1–6 (Oct 2016)
2. Bhowmik, A.K., Khendek, F., Hormati, M., Glitho, R.: An architecture for M2M enabled social networks. In: 2015 14th Annual Mediterranean Ad Hoc Networking Workshop (MED-HOC-NET), pp. 1–8, June 2015
3. Butzin, B., Golatowski, F., Timmermann, D.: Microservices approach for the internet of things. In: 2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 1–6, September 2016
4. ETSI: ETSI M2M solution introduction. About (2014). http://www.etsi.org/images/files/Events/2014/201405_DGCONNECT_SmartM2MAppliances/ETSI_M2M_introduction_main.pdf
5. Liu, D., Zhu, H., Xu, C., Bayley, I., Lightfoot, D., Green, M., Marshall, P.: CIDE: an integrated development environment for microservices. In: 2016 IEEE International Conference on Services Computing (SCC), pp. 808–812, June 2016
6. Nadareishvili, I., Mitra, R., McLarty, M., Amundsen, M.: Microservice Architecture, 1st edn. O'Reilly Media, Inc., Sebastopol (2016)
7. oneM2M: M2M Functional Architecture. Technical specification (2015). http://www.onem2m.org/images/files/deliverables/TS-0001-Functional_Architecture-V1_6_1.pdf
8. Pandey, A.K., Vasishtha, A.K., Saxena, A.S.: Properties and interaction of object oriented software agent with system. In: 2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom), pp. 1141–1143, March 2016
9. Skocir, P., Maracic, H., Kusek, M., Jezic, G.: Data filtering in context-aware multi-agent system for machine-to-machine communication. In: Jezic, G., Howlett, R.J., Jain, L.C. (eds.) Agent and Multi-Agent Systems: Technologies and Applications. SIST, vol. 38, pp. 41–51. Springer, Cham (2015). doi:10.1007/978-3-319-19728-9_4
10. Xu, C., Zhu, H., Bayley, I., Lightfoot, D., Green, M., Marshall, P.: CAOPLE: a programming language for microservices SAAS. In: 2016 IEEE Symposium on Service-Oriented System Engineering (SOSE), pp. 34–43, March 2016